

Project Goals and Objectives

Gaze tracking has powerful applications ranging from assistive technology to media research. For example, it can enable someone with severe motor impairments such as ALS or paralysis from spinal cord injuries to interact with a computer, browse menus or websites and communicate. Such technology serves as a vital link connecting severely disabled patients with their caregivers and family members. In the field of media research, understanding the audience's visual attention to advertisements and media is crucial in determining airtime pricing for television shows and live events.

Modern gaze tracking applications rely on advances in computer vision and deep learning models. With respect to assistive technology, deploying such models on devices with tight memory and power constraints helps democratize and make this technology more accessible and affordable to those who absolutely need it to function as independently as possible. Further, having such models run independently on lightweight devices rather than data centers protects these users from being at the mercy of service outages from SaaS cloud providers. With respect to media research, there are companies marketing themselves to advertising agencies by claiming to have the greatest and highest quality data on audiences. Deploying gaze tracking models on lightweight devices enables them to collect even more data while keeping the costs of data collection down.

Our goal for this project was to build a TinyML application that can capture a picture of someone's face and determine whether the person is looking left with respect to the camera, straight at the camera or right with respect to the camera. The underlying machine learning model is a three-way image classification application. In terms of hardware, we restricted our scope to using the Arduino Nano 33 BLE Sense board provided in class. This device came equipped with 1 MB of Flash memory and 256 kB of RAM. We also had a TinyML Shield carrier board that allowed our Arduino board to be connected to an OV7675 Camera Module. The final product envisioned was an Arduino application that captures a grayscale image of someone's face when the TinyML Shield button is pressed and then has its deployed machine learning model predict which way that person was looking - left, straight, or right.

Training Data

The training image data came from two publicly available sources. The first source was the Kaggle Gaze Capture (20GB) dataset [1]. This dataset contained thousands of annotated images taken from iPhone selfies of about 260 subjects. The second source was the dataset available from the MPIIFaceGaze project [2]. The MPIIFaceGaze dataset also contained thousands of annotated images from 15 subjects. Both these sources have annotations that allow us to compute a yaw value for an image and to classify it as left, straight or right. However, there are some key differences in the annotations available between these two sources.

Processing GazeCapture Images

The GazeCapture dataset from Kaggle comes packaged in a 20 GB zip file. Once unpacked, the package contains folders for each subject named with a 5 digit `<subject_id>`. All images and metadata for each subject is contained in `<root>/<subject_id>/<subject_id>`. It should contain the following metadata files

- `info.json`
- `frames.json`
- `appleFace.json`
- `dotInfo.json`
- `appleLeftEye.json`
- `appleRightEye.json`
- `motion.json`

It also contains another folder called `frames` that contains indexed jpeg images starting from 00000. For example, subject 00173's fifth image would be contained in

`<root>/00173/00173/frames/00004.jpg`.

First, we do an 80-20 *subject level* train-test split. Then for each image of the subject, we look at the `IsValid` flags for that image in `appleFace.json`, `appleLeftEye.json` and `appleRightEye.json`. All three must have a value of 1 to be included.

We also had an additional motion filter based on data in `motion.json` and `dotInfo.json`. For a given image, we look up the image's `DotNum` and `Time` from `dotInfo.json`. We then find the closest time-stamped entry in `motion.json` for that same `DotNum` and compute the magnitude of the three dimensional `UserAcceleration` and `RotationRate` vectors. We include images whose `UserAcceleration` magnitude falls below 0.08g and whose `RotationRate` magnitude falls below 0.3 rad/s.

We then use the data in `appleFace.json` to build the crop. In this file, each image has four values: `X`, `Y`, `W`, `H`. The `X`, `Y` values represent the coordinates for the upper left corner of the image while `W`, `H` represent the pixel width and height of the bounding box for the face. We compute the following:

```
left = max(0, X - 0.15*W)
top = max(0, Y - 0.15*H)
right = min(224, X + 1.15*W)
bottom = min(224, Y + 1.15*H)
```

This leaves a 15% margin for the crop. We exclude the image if `right<=left` or `bottom<=top`. Finally we crop using `(left, top, right, bottom)` and rescale the image to a 224 X 224 pixel jpeg image using Lanczos resampling.

The image then gets classified based on its `XCam` value in `dotInfo.json` in the following manner

- If `abs(XCam) <= 3.0 cm`, label the image as straight
- If `XCam > 3.0 cm`, label the image as right
- If `XCam < -3.0 cm`, label the image as left

Subjects in this dataset were told to look at a dot on a screen, and `XCam` is the on-screen dot's position in centimeters, relative to the camera lens.

Processing MPIIFaceGaze Images

The MPIIFaceGaze data comes in a 950 MB ZIP file. When unpackaged, there are 15 folders (`p00-p14`) corresponding to each subject. Within each folder there is a text file containing annotation metadata (e.g. `p03/p03.txt`) along with other sub-folders (`day__`) each containing a series of JPEG images.

We first do a ***subject-level*** 80/20 train/test split. In a subject's annotation file, each line has the following 28 fields with 0 based indexing:

```
0      image path, relative to the subject folder (e.g. day01/0005.jpg)
1-2    on-screen gaze point, in pixels (unused here)
3-14   6 facial landmarks (x,y) - 4 eye corners + 2 mouth corners
15-20  estimated 3D head pose
21-23  fc: 3D face center in the camera coordinate system
24-26  gt: 3D gaze target location in the camera coordinate system
27     which eye was used in the original paper's eval subset
```

Next we build the crop parameters from the 6 facial landmarks. We compute `min_x`, `max_x`, `min_y`, `max_y` from the facial landmarks and define `w = max_x - min_x` and `h = max_y - min_y`.

We then compute

```
left = max(0, min_x - 0.55 * w)
right = min(224, max_x + 0.55 * w)
top = max(0, min_y - 1.2 * h)
bottom = min(224, max_y + 0.75 * h)
```

We exclude the image if `right<=left` or `bottom<=top`. Finally we crop using (`left`, `top`, `right`, `bottom`) and rescale the image to a 224 X 224 pixel jpeg image using Lanczos resampling.

The image gets classified using the `fc` and `gt` vectors. We compute the yaw in degrees using this function

```
def gaze_to_yaw(fc, gt):
    g = [gt[i] - fc[i] for i in range(3)]
    norm = math.sqrt(sum(v * v for v in g))
    if norm == 0:
        return 0.0
    gx, gy, gz = (v / norm for v in g)
    return math.degrees(math.atan2(-gx, -gz))
```

The image gets labeled in the following manner.

- If `abs(gaze_to_yaw(fc, gt)) <= 5.0` degrees, label the image as straight
- If `gaze_to_yaw(fc, gt) > 5.0` degrees, label the image as right
- If `gaze_to_yaw(fc, gt) < -5.0` degrees, label the image as left

Training and Test Dataset Class Counts

The processed 224 X 224 jpeg training and test images from the GazeCapture and MPIIFaceGaze datasets were combined. We have the following class counts

```
--- Training Dataset Class Counts ---
```

```
--- Sample Counts per Class ---
```

```
Class 'left': 49839 samples
```

```
Class 'right': 53019 samples
```

```
Class 'straight': 71028 samples
```

```
--- Test Dataset Class Counts ---
```

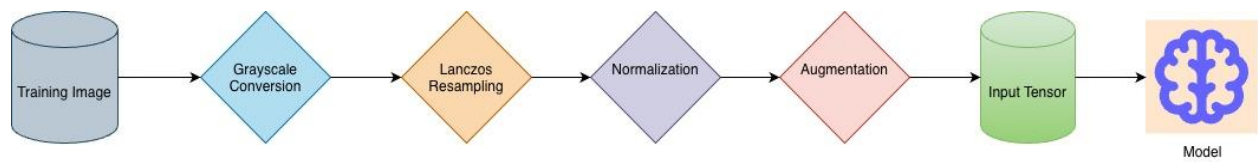
```
--- Sample Counts per Class ---
```

```
Class 'left': 8923 samples
```

```
Class 'right': 9529 samples
```

```
Class 'straight': 13781 samples
```

Training Pipeline



The figure above shows a high-level overview of how our training and test images were fed into our model. All images were given as colored 224 X 224 pixel JPEG files. When training our models, 20 percent of the training dataset was set aside for validation.

Grayscale Conversion and Rescaling

We used the `tf.keras.utils.image_dataset_from_directory` function to convert the input images into grayscale and resize them using Lanczos resampling. They were either resized into 64X64 8-bit grayscale pixel images or 48X48 8-bit grayscale pixel images.

Normalization

Each pixel was converted from an 8-bit value into a floating point value ranging from 0.0 to 1.0. This helps in training the model by making the gradients smoother. The function below shows how we normalized images

```
def normalize_images(image, label):  
    # This function now receives a single image (3-D tensor: H, W, C)  
    image = tf.cast(image, tf.float32) / 255.0  
    return image, label
```

Augmentation

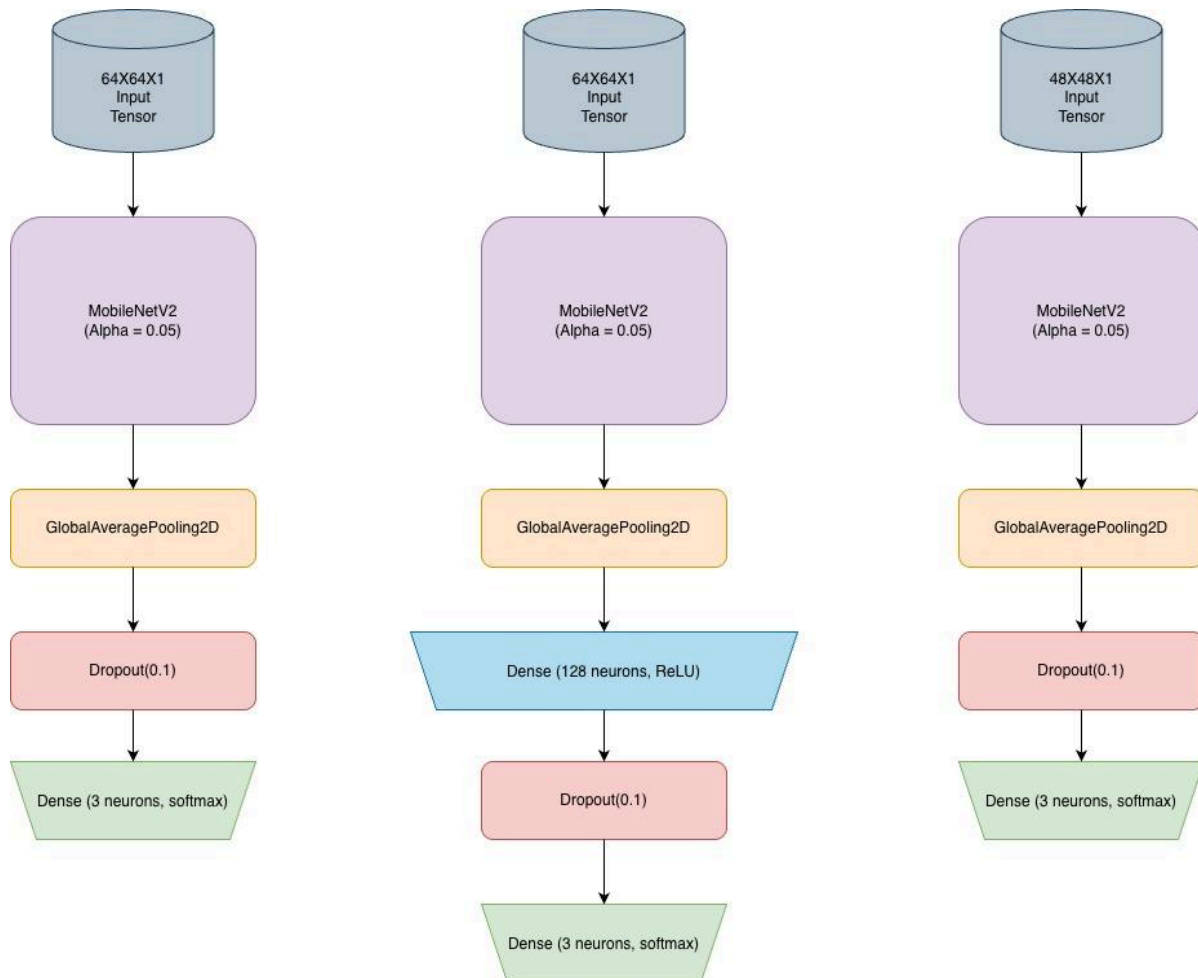
Data augmentation is a technique that can help improve the accuracy of our models by introducing additional artificial data [3]. For our training images, we made small random changes during the training process. Introducing these changes helps prevent the model from taking shortcuts by overfitting on superficial clues in the training data. It also helps the model handle device noise from our camera as well as different lighting conditions. This modification was applied only to the training images, not the validation or test images.

We augmented our images using the following function:

```
def augment_image(image, label):
    # Increase image size by random scale factor, and crop it back to
    # original dimensions randomly
    resize_factor = tf.random.uniform(
        [],
        minval=1.0,
        maxval=1.2,
        dtype=tf.float32
    )
    new_height = tf.cast(resize_factor * INPUT_SHAPE[0], tf.int32)
    new_width = tf.cast(resize_factor * INPUT_SHAPE[1], tf.int32)
    image = tf.image.resize_with_crop_or_pad(image, new_height, new_width)
    image = tf.image.random_crop(image, size=INPUT_SHAPE)

    # Randomly change the brightness of the image
    image = tf.image.random_brightness(image, max_delta=0.2)
    return image, label
```

Model Development



The figure above is a high-level diagram of the three models we tested.

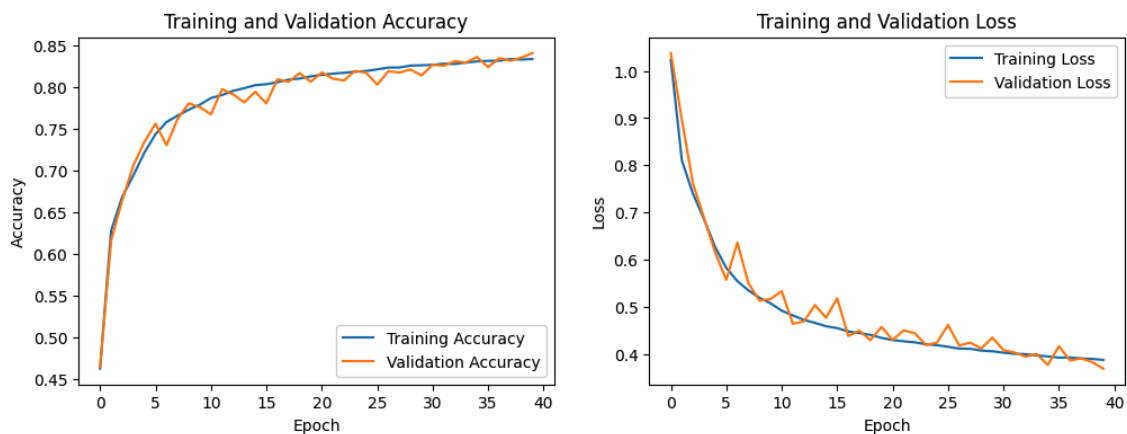
The left model (Model 1) has a 64 X 64 X 1 input tensor followed by a MobileNetV2 model with $\alpha = 0.05$, followed by a GlobalAveragePooling2D layer, followed by a Dropout(0.1) layer followed by a Dense 3-neuron softmax layer.

The center model (Model 2) has a 64 X 64 X 1 input tensor followed by a MobileNetV2 model with $\alpha = 0.05$, followed by a GlobalAveragePooling2D layer, followed by a Dense 128 neuron ReLU layer, followed by a Dropout(0.1) layer, followed by a Dense 3-neuron softmax layer. This is a transfer learning model where the weights for the MobileNetV2 model were frozen with pre-trained weights from another MobileNetV2 $\alpha=0.05$ model with a 96 X 96 X 1 input grayscale tensor trained on ImageNet [4]. This follows a similar approach from our Mask Detection lab. The main difference is the Mask Detection transfer learning model was built using Edge Impulse using their proprietary weights and with a different input tensor for color images.

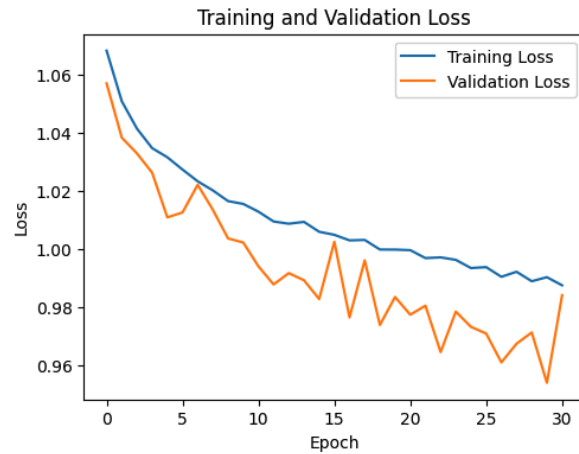
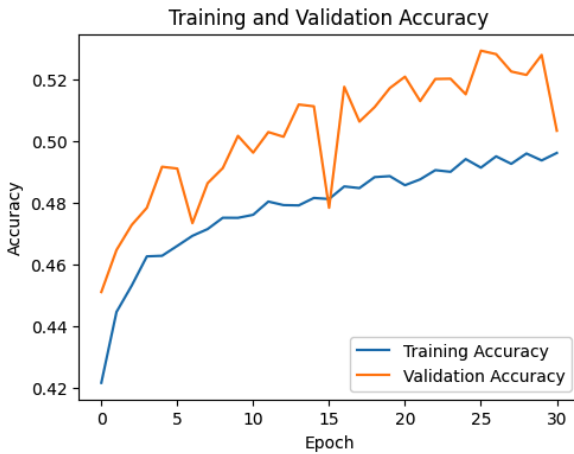
The right model (Model 3) has a 48 X 48 X 1 input tensor followed by a MobileNetV2 model with $\alpha = 0.05$, followed by a GlobalAveragePooling2D layer, followed by a Dropout(0.1) layer followed by a Dense 3-neuron softmax layer. This model was developed in case our Arduino board did not have enough RAM to accommodate 64 X 64 X 1 grayscale input tensors.

Model Training

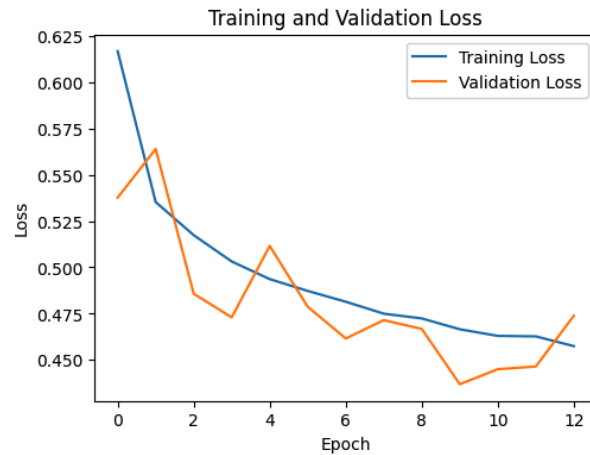
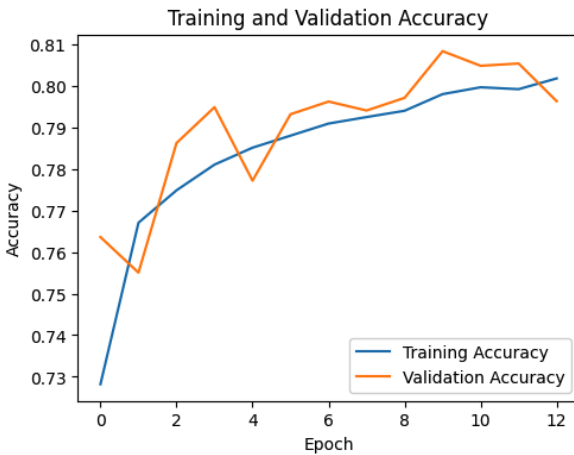
All three models were trained using the Google Colab environment and its T4 GPU Runtime. For TinyML deployment, all models would be then quantized into INT8.



Model 1: This model was trained for 40 epochs with a batch size of 32 and a 5 epoch early stopping callback from scratch with no pre-trained weights. It was trained using the ADAM optimizer with the default learning rate, a `sparse_categorical_crossentropy` loss function and `accuracy` as a metric. Total parameters: 56,019 (218.82 KB)



Model 2: This model was trained for 40 epochs with a batch size of 32 and a 5 epoch early stopping callback. It was trained using the ADAM optimizer with the default learning rate, a `sparse_categorical_crossentropy` loss function and `accuracy` as a metric. Total parameters: 216,531 (845.82 KB)



Model 3: Starting with the weights for MobileNetV2 from Model 1, this model was fine-tuned for 15 epochs with a batch size of 32 and a 3 epoch early stopping callback. It was trained using the ADAM optimizer with the default learning rate, a `sparse_categorical_crossentropy` loss function and `accuracy` as a metric. Total parameters: 56,019 (218.82 KB)

For TinyML deployment, we favored Model 1. Model 2 - the transfer learning model - performed worse even with weights pre-trained on ImageNet. Further, the 128-neuron Dense layer following the MobileNetV2 model and GlobalAveragePooling2D layer introduces more parameters to the model and thus requires more memory.

System Design and On-Device Deployment:

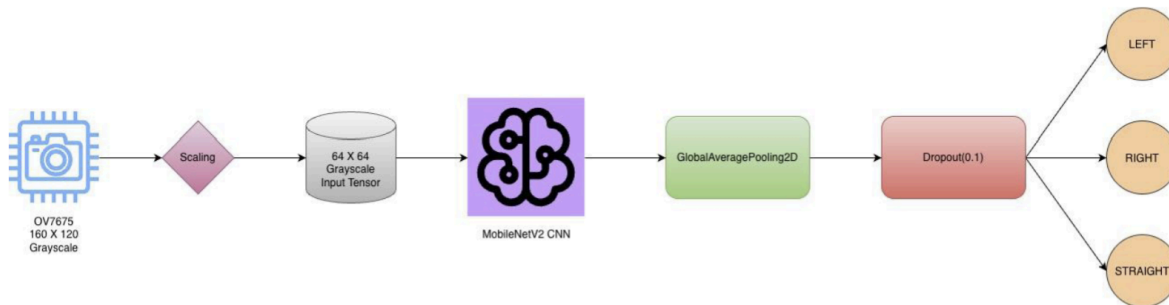


Figure: System Architecture Pipeline

We used an OV7675 camera module with the Arduino Nano to capture a 160 by 120 pixel grayscale image rather than RGB. After we trained our model and converted it to int8, as the Nano because it has limited memory, and TFLite Micro requires it to be int8-scaled too. So the Nano performs preprocessing by center-cropping the frame to 64 by 64 pixels, rounding each pixel value from 0 to 1, and then putting them into a tensor arena. To optimize Flash utilization while maintaining full support for MobileNetV2 operations, we employed MicroMutableOpResolver configured with 12 specific operators: Conv2D, DepthWiseConv2D, FullyConnected, Softmax, Mean, AveragePool2D, Reshape, add/pad, quantize/dequantize, and Relu. The TFLite Micro interpreter allocates a 150 KB tensor arena, and the camera buffer consumes approximately 19 KB of RAM. Final compilation reports indicate a Flash footprint of roughly 353.81 KB (around 35% of flash memory) and total RAM consumption of 220.47 KB, which is around 84% of the total SRAM available to the board. On-device metrics indicate that capturing an image takes around 1.4 s, while model inference requires 250-400 ms. A screenshot of the on-device

```
[Board] [Taking Snapshot...]  
[Board] >>> STEP 1: IMAGE QUALITY  
[Board] Capture Time: 1444 ms  
[Board] Avg Brightness: 108 / 255  
Photo captured and displayed.  
[Board] >>> STEP 2: ML CLASSIFICATION  
[Board] Inference Time: 372 ms  
[Board] - left: 0.0000  
[Board] - right: 0.7891  
[Board] - straight: 0.2070  
[Board] WINNING CLASSIFICATION: -> right <-
```

Figure: On-Device Inference

The camera captures the grayscale frame and sends the image, and is triggered by pressing the button on the shield using a byte-aligned serial protocol. Each message begins with a header to allow the listener to stay aligned even after the data is lost. After the header, a message type byte is transferred with image packet containing the raw 160*120 image through the Python listener and displayed with OpenCV. String packets include a length byte followed by text data for status updates or debug output. This lightweight structure makes it easy for the laptop to reliably decode and visualize incoming frames while also receiving readable messages.

Results and Analysis:

Baseline Model Performance

Model	Train Accuracy	Validation Accuracy	Test Accuracy	Size
Model 1: MobileNetV2_Alpha0.05_64X64	0.8343	0.8413	0.7002	218.82 KB
Model 2: MobileNetV2_Alpha0.05_64X64 (Transfer Learning)	0.4913	0.5292	0.394	845.82 KB
Model 3: MobileNetV2_Alpha0.05_48X48 (Fine Tuning)	0.7979	0.8083	0.6949	218.82 KB

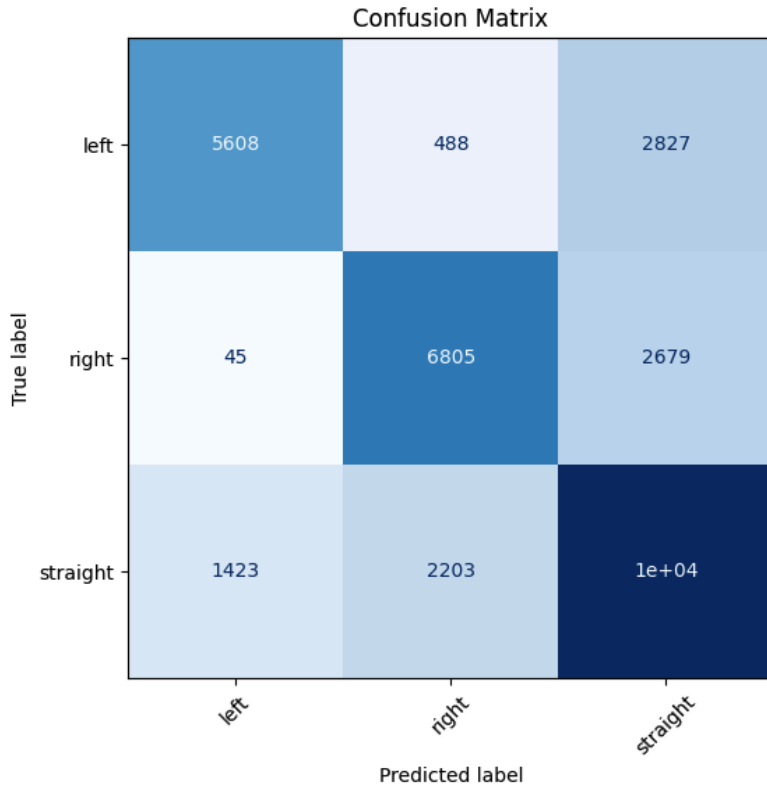
Among the uncompressed models, Model 1 was selected for compression and deployment. Model 2 showed relatively poorer performance while requiring more memory.

Uncompressed Deployed Model Classification Report

Test Accuracy: 0.7002

Classification Report:

	precision	recall	f1-score	support
left	0.7925	0.6285	0.7010	8923
right	0.7166	0.7141	0.7154	9529
straight	0.6484	0.7369	0.6898	13781
accuracy			0.7002	32233
macro avg	0.7192	0.6932	0.7021	32233
weighted avg	0.7085	0.7002	0.7005	32233



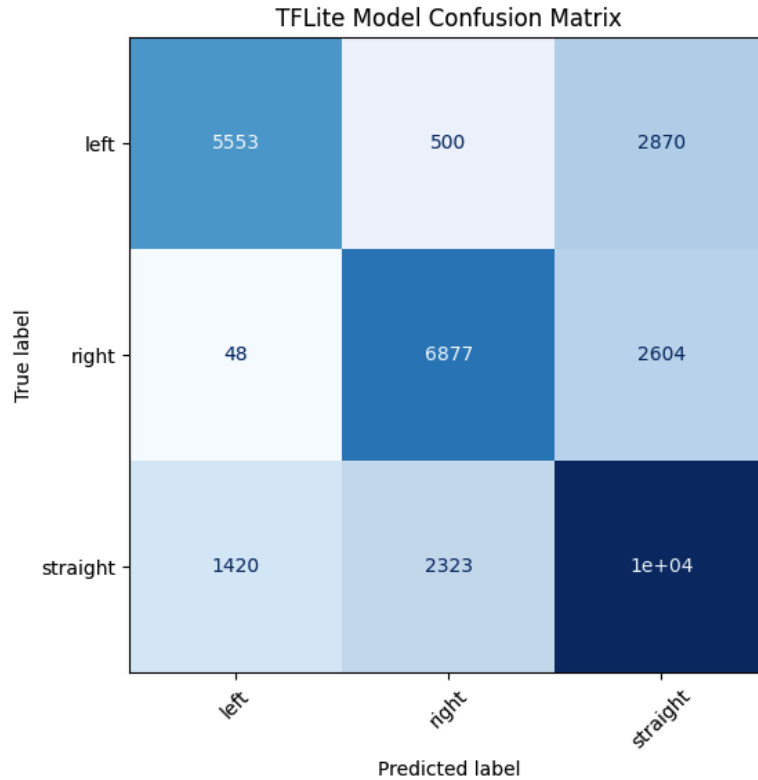
Compressed Model Performance

Model	Test Accuracy	TFLite Model Size
Model 1: MobileNetV2_Alpha0.05_64X64	0.697	168.39 KB
Model 2: MobileNetV2_Alpha0.05_64X64 (Transfer Learning)	0.3828	320.73 KB
Model 3: MobileNetV2_Alpha0.05_48X48 (Fine Tuning)	0.6912	168.45 KB

Compressed Deployed Model Classification Report

Test Accuracy: 0.6970

	precision	recall	f1-score	support
left	0.7909	0.6223	0.6966	8923
right	0.7090	0.7217	0.7153	9529
straight	0.6471	0.7284	0.6854	13781
accuracy			0.6970	32233
macro avg	0.7157	0.6908	0.6991	32233
weighted avg	0.7052	0.6970	0.6973	32233



Overall, there was no significant drop in accuracy with post-training quantization for all three models. We saw no need for any quantization aware training. Even if the accuracy of the deployed model (Model 1) was around 69.7%, we notice from the confusion matrix above that the vast majority of missed predictions involved borderline cases between straight and left/right. This indicates that the model misses more often in cases where it is hard to distinguish between left/right and straight. But the model rarely predicted left on a true right image or right on a true left image.

Device Performance results:

We collected 20 samples of each class of the classification model in average room light settings. We tried to adjust the right orientation and distance from someone's face to the camera to capture more facial features for our snapshot on the camera. But due to camera quality and very limited memory allocation to process the picture and perform model inference. So we couldn't get close to the accuracy rate from our model testing on a Google Colab notebook. So we received results from 45% to 90% from the table shown below, with the confusion matrix of 10 samples each from one person and a total of 60 samples for the three cases.

State	On device Accuracy
Straight	50%
Left	45%
Right	90%

Figure: Accuracy of each state from on-device sampling

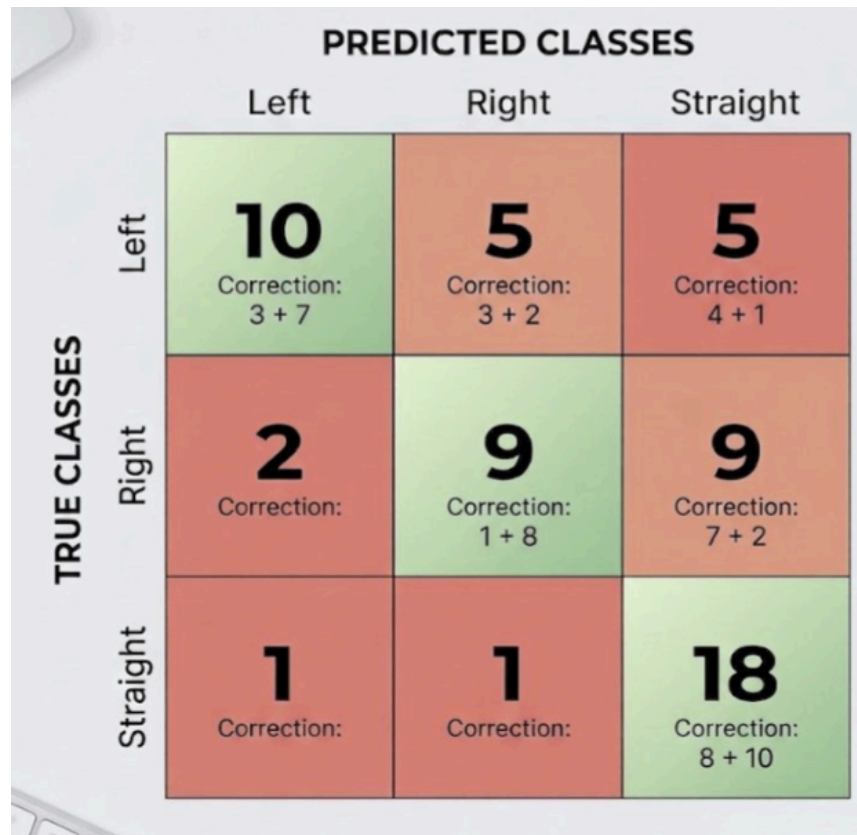


Figure: Classification/Accuracy matrix for Device performance

Challenges, Limitations, and Future work:

Training data played a critical role in the performance of the model, as both labeling accuracy and overall data quality directly influenced the results. Ensuring an equal number of samples for each state helped maintain balanced learning, while preprocessing significantly improved the effectiveness of the dataset and supported higher model accuracy. We also attempted at first to create a 5-way classification application. However, the data sources did not have enough pitch data to make a good classifier. We suspect this is due to the fact that

GazeCapture images were collected from Apple iPhones and most people have a tendency to look down on their phones rather than up.

One must also take into serious consideration the different ways images were labeled from the GazeCapture and MPIIFaceGaze datasets. GazeCapture images were classified according to the horizontal displacement of a target dot from the camera's point of view. It did not take into account how centered the subject's face was with respect to the camera. On the other hand, MPIIFaceGaze images were classified according to the yaw of a computed gaze vector determined from facial and gaze target coordinates.

In terms of on-device performance, we were very limited by the quality of the OV7675 camera we had to work with. We had problems taking pictures that could capture all our facial features. We could have attempted to adjust the camera lens focus. But doing so was extremely difficult without proper tools and we really did not want to risk damaging the camera that was lent to us.

Looking ahead, the project's limitations highlight how constrained hardware resources shape every aspect of a vision system. Future improvements include upgrading to a camera module with better image quality and lower distortion, as well as using a platform with more memory, such as a Raspberry Pi, to support larger models with higher accuracy and additional classification states. We could also explore more sophisticated labeling schemes for our training data.

References

- [1] <https://www.kaggle.com/datasets/dhruv413/gaze-capture-20gb-zip>
- [2] <https://www.collaborative-ai.org/research/datasets/MPIIFaceGaze/>
- [3] <https://mlsysbook.ai/kits/assets/downloads/Hardware-Kits.pdf>, page 72
- [4] https://github.com/NavodPeiris/MobileNet_96x96_greyscale_weights